Weather App — komunikacja sieciowa w PWA

Na tych zajęciach zbudujemy w **React** aplikację pogodową, która pobiera dane z internetowego API (**OpenWeatherMap**) i wyświetla pogodę dla wpisanego miasta. Następnie zrobimy z niej aplikację **PWA**, która po utracie połączenia pokazuje ostatnio pobraną pogodę. Poznamy:

- wysyłanie zapytań HTTP funkcją `fetch()` i obsługę błędów (nieznane miasto, brak sieci),
- stan komponentu w React (`useState`),
- buforowanie odpowiedzi API w service workerze (strategia „najpierw sieć").

Kod oddajemy w repozytorium w folderze `lab_03/` (patrz „Oddanie" na końcu).

Przygotowanie

W pracowni Node.js jest już zainstalowany — sprawdzamy tylko jego wersję (narzędzie Vite wymaga wersji 20.19 lub nowszej, a najlepiej aktualnej LTS):

```
node --version
```

Praca w domu: jeśli polecenie nie działa albo wersja jest starsza, instalujemy aktualną wersję LTS ze strony https://nodejs.org/en/download/ (po instalacji trzeba otworzyć terminal na nowo).

Klucz API OpenWeatherMap

Dane o pogodzie pobierzemy z serwisu OpenWeatherMap, który wymaga **klucza API** (identyfikatora aplikacji):

1. Zakładamy darmowe konto na https://home.openweathermap.org/users/sign_up i potwierdzamy adres e-mail.
2. Klucz znajdziemy w zakładce **API keys** (https://home.openweathermap.org/api_keys) — kopiujemy go.

Korzystamy z usługi **Current Weather Data** (bieżąca pogoda), która jest dostępna w darmowym planie — domyślny klucz utworzony przy rejestracji wystarcza. **Nie wybieramy żadnego planu ani subskrypcji i nie podajemy karty płatniczej.** Usługa **One Call API 3.0**, mocno reklamowana na stronie z cennikiem, wymaga osobnej subskrypcji z kartą — nie jest nam potrzebna.

Nowy klucz zaczyna działać dopiero **po pewnym czasie** (zwykle kilkadziesiąt minut, czasem do kilku godzin) — do tego czasu serwis odpowiada błędem **401**. Najlepiej założyć konto przed zajęciami. Jeśli nasz klucz jeszcze nie działa, na czas zajęć korzystamy z klucza podanego przez prowadzącego: `f33a484cf794d08d0148764789aaba32`, a swój wpisujemy później.

Czy klucz już działa, sprawdzimy, otwierając w przeglądarce adres (w miejsce `KLUCZ` wklejamy swój klucz):

```
https://api.openweathermap.org/data/2.5/weather?q=siedlce&appid=KLUCZ
```

Jeśli zobaczymy dane w formacie JSON — klucz działa; komunikat z kodem `401` (`Invalid API key`) oznacza, że klucz jeszcze nie jest aktywny.

Tworzenie projektu

Tworzymy folder `lab_03` (bez spacji i wielkich liter w nazwie — tak będzie się nazywał projekt), **poza folderami synchronizowanymi przez OneDrive, Dropbox itp.** (na Windowsie często dotyczy to Pulpitu i Dokumentów) — np. `C:\dev\lab_03`. Program synchronizujący blokuje pliki w `node_modules` i Vite zgłasza błędy (`EBUSY: resource busy or locked`). Otwieramy w nim terminal i tworzymy aplikację React narzędziem **Vite**:

```
npm create vite@latest . -- --template react
```

Kropka oznacza „w bieżącym folderze". Przy pierwszym uruchomieniu `npm` zapyta o zgodę na pobranie pakietu `create-vite` — potwierdzamy. Na pytanie o linter (**Which linter to use?**) zostawiamy odpowiedź domyślną (Enter), a na **Install with npm and start now?** odpowiadamy **Yes** — Vite zainstaluje zależności i uruchomi serwer deweloperski. W terminalu pojawi się adres (np. `http://localhost:5173/`) — otwieramy go w Chrome. Serwer zatrzymujemy **Ctrl+C**, a uruchamiamy ponownie poleceniem:

```
npm run dev
```

W starszych poradnikach aplikacje React tworzy się poleceniem `npx create-react-app` — to narzędzie zostało wycofane (2025) i nie działa poprawnie z aktualnym Reactem.

Vite utworzył m.in.:

- `index.html` — strona z pustym elementem `<div id="root">`, do którego React wstawi aplikację,
- `src/main.jsx` — punkt startowy: tworzy aplikację w elemencie `#root` (`createRoot(...).render(...)`),
- `src/App.jsx` — główny komponent aplikacji (pliki z kodem JSX mają w Vite rozszerzenie `.jsx`),
- `src/App.css`, `src/index.css` — style.

Porządkujemy projekt:

1. W pliku `src/main.jsx` usuwamy linijkę `import './index.css'` i **zapisujemy plik**, a dopiero potem kasujemy plik `src/index.css` (jego style gryzłyby się z naszymi). Jeśli w konsoli przeglądarki zostanie błąd `404` dla `index.css`, odświeżamy stronę (**Ctrl+F5**).
2. Zawartość pliku `src/App.css` zastępujemy zawartością pliku **`App.css`** z materiałów do zajęć.
3. Zawartość pliku `src/App.jsx` zastępujemy prostym komponentem:

```jsx
function App() {
  return <h1>App</h1>;
}

export default App;
```

Po zapisaniu plików strona w przeglądarce odświeży się sama i pokaże napis **App** — na razie na białym tle, bez naszych stylów (plik `App.css` podłączymy w następnym kroku).

Pobieranie danych z API

Tworzymy folder `src/api`, a w nim plik `fetchWeather.js` — tu umieścimy całą komunikację z serwisem pogodowym (w miejsce `TU_WKLEJ_KLUCZ_API` wklejamy swój klucz):

```
const URL = 'https://api.openweathermap.org/data/2.5/weather';
const API_KEY = 'TU_WKLEJ_KLUCZ_API';

export async function fetchWeather(query) {
  const params = new URLSearchParams({
    q: query.trim().toLowerCase(), // nazwa miasta
    units: 'metric', // stopnie Celsjusza, m/s
    lang: 'pl', // opis pogody po polsku
    appid: API_KEY,
  });

  let response;
  try {
    response = await fetch(`${URL}?${params}`);
  } catch {
    // fetch() zgłasza błąd, gdy w ogóle nie udało się połączyć z serwerem
    throw new Error('Brak połączenia z internetem.');
  }

  if (!response.ok) {
    // serwer odpowiedział, ale z kodem błędu (np. 404 - nie ma takiego miasta)
    if (response.status === 404) throw new Error('Nie znaleziono miasta.');
    if (response.status === 401) throw new Error('Nieprawidłowy lub nieaktywny klucz API.');
    throw new Error(`Błąd serwera pogody (${response.status}).`);
  }

  return response.json();
}
```

Co tu się dzieje:

- `URLSearchParams` składa parametry zapytania w adres, np. `...weather?q=siedlce&units=metric&lang=pl&appid=...` (i poprawnie koduje polskie znaki),
- `await fetch(adres)` wysyła zapytanie HTTP GET i czeka na odpowiedź,
- `fetch()` zgłasza wyjątek **tylko** wtedy, gdy nie da się połączyć (brak sieci). Gdy serwer odpowie błędem (np. **404** dla nieznanego miasta), `fetch()` kończy się normalnie — dlatego sami sprawdzamy `response.ok` (`true` dla kodów 200–299),
- `response.json()` zamienia treść odpowiedzi (JSON) na obiekt JavaScript,
- słowo `export` udostępnia funkcję innym plikom.

Nazwę miasta zamieniamy na małe litery, żeby „Siedlce" i „siedlce" dawały ten sam adres zapytania — przyda się to przy pracy offline.

W starszych poradnikach do zapytań HTTP używa się biblioteki **axios** (`axios.get(...)`). Działa podobnie, ale od kilku lat wbudowane `fetch()` jest dostępne we wszystkich przeglądarkach i w Node.js, więc dodatkowa biblioteka nie jest potrzebna.

Zawartość `src/App.jsx` zastępujemy kodem z polem wyszukiwania:

```
import { useState } from 'react';
import { fetchWeather } from './api/fetchWeather.js';
import './App.css';

function App() {
  const [query, setQuery] = useState('');

  const search = async (e) => {
    e.preventDefault(); // formularz nie przeładowuje strony
    const data = await fetchWeather(query);
    console.log(data);
  };

  return (
    <div className="main-container">
      <form onSubmit={search}>
        <input
          type="text"
          className="search"
          placeholder="Wpisz miasto..."
          value={query}
          onChange={(e) => setQuery(e.target.value)}
        />
      </form>
    </div>
  );
}

export default App;
```

Do komponentu dodaliśmy:

- **stan** `query` — `useState('')` zwraca aktualną wartość (na początku pusty napis) i funkcję `setQuery`, która ją zmienia. Każda zmiana stanu powoduje ponowne narysowanie komponentu,
- pole tekstowe (`input`) z klasą `search`, którego wartość jest powiązana ze stanem: `value={query}` wyświetla stan, a `onChange` zapisuje w nim każdą zmianę (`e.target` to element, który wywołał zdarzenie — tu nasze pole),
- formularz (`form`) — zdarzenie `onSubmit` wywołuje się po naciśnięciu **Enter** (a na telefonie po naciśnięciu przycisku „Szukaj”/„Idź” na klawiaturze). `e.preventDefault()` blokuje domyślne przeładowanie strony po wysłaniu formularza,
- funkcję `search`, która pobiera pogodę i na razie wypisuje ją w konsoli.

Wpisujemy nazwę miasta (np. `Siedlce`), naciskamy **Enter** i sprawdzamy wynik w konsoli (**F12** → **Console**) — to obiekt z danymi o pogodzie:

```
App.js:13
▼ {coord: {…}, weather: Array(1), base: 'stations', main: {…}, visibility: 10000, …} ℹ
    base: "stations"
  ▶ clouds: {all: 96}
    cod: 200
  ▶ coord: {lon: 22.2901, lat: 52.1677}
    dt: 1642704103
    id: 759412
  ▶ main: {temp: -1.64, feels_like: -8.64, temp_min: -1.64, temp_max: -1.…
    name: "Siedlce"
  ▶ sys: {type: 2, id: 2043185, country: 'PL', sunrise: 1642660072, sunse…
    timezone: 3600
    visibility: 10000
  ▶ weather: [{…}]
  ▶ wind: {speed: 8.91, deg: 278, gust: 15.41}
  ▶ [[Prototype]]: Object
```

Najważniejsze pola: `name` (nazwa miasta), `sys.country` (kod kraju), `main.temp` (temperatura), `main.pressure` (ciśnienie), `main.humidity` (wilgotność), `weather[0].description` i `weather[0].icon` (opis i ikona pogody), `wind` (wiatr), `dt` (czas pomiaru). W zakładce **Network** możemy zobaczyć samo zapytanie do `api.openweathermap.org` i odpowiedź serwera.

Wpiszmy teraz nieistniejące miasto (np. `xyzabc`) — w konsoli pojawi się nieobsłużony błąd **Nie znaleziono miasta.** Obsłużymy go w następnym kroku.

Wyświetlanie pogody

Zamiast wypisywać dane w konsoli, zapiszemy je w stanie komponentu i wyświetlimy. Dodamy też stan dla komunikatu o błędzie i dla trwającego pobierania. Zawartość `src/App.jsx` zastępujemy:

```jsx
import { useState } from 'react';
import { fetchWeather } from './api/fetchWeather.js';
import './App.css';

function App() {
  const [query, setQuery] = useState('');
  const [weather, setWeather] = useState(null);
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const search = async (e) => {
    e.preventDefault();
    if (!query.trim()) return; // puste pole - nic nie robimy
    setLoading(true);
    setError('');
    try {
      const data = await fetchWeather(query);
      setWeather(data);
      setQuery(''); // czyścimy pole po udanym wyszukiwaniu
```

```
      } catch (err) {
        setWeather(null);
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    return (
      <div className="main-container">
        <form onSubmit={search}>
          <input
            type="text"
            className="search"
            placeholder="Wpisz miasto..."
            value={query}
            onChange={(e) => setQuery(e.target.value)}
          />
        </form>

        {loading && <p className="message">Pobieranie danych...</p>}
        {error && <p className="message error">{error}</p>}

        {weather && (
          <div className="city">
            <h2 className="city-name">
              <span>{weather.name}</span>
              <sup>{weather.sys.country}</sup>
            </h2>
          </div>
        )}
      </div>
    );
  }

  export default App;
```

Zapis `{weather && ( ... )}` działa tak: jeśli `weather` jest „puste" (`null`), React niczego nie wyświetla; jeśli zawiera dane, wyświetla element po prawej stronie `&&`. Tak samo działają komunikaty `loading` i `error`. Blok `try ... catch ... finally` łapie błędy rzucone w `fetchWeather()` (nieznane miasto, brak sieci, zły klucz) i wyświetla ich treść na stronie, a `finally` wykonuje się zawsze — także po błędzie.

Sprawdzamy: `Siedlce` → karta z nazwą miasta i kodem kraju, `xyzabc` → komunikat **Nie znaleziono miasta.**

Aplikacja PWA — praca bez sieci

Z aplikacji zrobimy PWA za pomocą wtyczki **vite-plugin-pwa**. Wtyczka korzysta z biblioteki Workbox (tej samej co w lab01): sama generuje service workera, który zapisuje w cache pliki aplikacji, oraz plik manifestu. My dopiszemy tylko, jak buforować **odpowiedzi API**.

Zatrzymujemy serwer (**Ctrl+C**) i instalujemy wtyczkę:

```
npm install -D vite-plugin-pwa
```

Ostrzeżenia `npm warn deprecated ...` dotyczą bibliotek używanych przez wtyczkę — możemy je zignorować.

Ikony aplikacji wygenerujemy z logo Vite (`public/favicon.svg`) — możemy je wcześniej podmienić na własny obrazek SVG lub PNG (wtedy w poleceniu podajemy jego nazwę):

```
npx @vite-pwa/assets-generator --preset minimal-2023 public/favicon.svg
```

W folderze `public` pojawią się pliki `pwa-64x64.png`, `pwa-192x192.png`, `pwa-512x512.png`, `maskable-icon-512x512.png`, `apple-touch-icon-180x180.png` i `favicon.ico`. Generator wypisze też w terminalu znaczniki `<link ...>` i listę `icons` — nie kopiujemy ich, ikony dopiszemy w konfiguracji poniżej.

Zawartość pliku `vite.config.js` (w głównym folderze projektu) zastępujemy:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: 'autoUpdate', // nowa wersja aplikacji instaluje się sama
      manifest: {
        name: 'Pogoda',
        short_name: 'Pogoda',
        description: 'Aktualna pogoda dla wybranego miasta',
        lang: 'pl',
        display: 'standalone',
        background_color: '#1e3c72',
        theme_color: '#1e3c72',
        icons: [
          { src: 'pwa-64x64.png', sizes: '64x64', type: 'image/png' },
          { src: 'pwa-192x192.png', sizes: '192x192', type: 'image/png' },
          { src: 'pwa-512x512.png', sizes: '512x512', type: 'image/png' },
          { src: 'maskable-icon-512x512.png', sizes: '512x512', type: 'image/png', purpose:
'maskable' },
        ],
      },
      workbox: {
        runtimeCaching: [
          {
            // odpowiedzi API: najpierw sieć, a bez sieci - ostatnia zapisana odpowiedź
            urlPattern: ({ url }) => url.origin === 'https://api.openweathermap.org',
            handler: 'NetworkFirst',
            options: {
              cacheName: 'weather-api',
              networkTimeoutSeconds: 5,
              expiration: { maxEntries: 20, maxAgeSeconds: 24 * 60 * 60 },
            },
          },
```

```
          {
            // ikony pogody: najpierw cache (ikony się nie zmieniają)
            urlPattern: ({ url }) => url.origin === 'https://openweathermap.org' &&
url.pathname.startsWith('/img/'),
            handler: 'CacheFirst',
            options: {
              cacheName: 'weather-icons',
              expiration: { maxEntries: 50 },
            },
          },
        ],
      },
    }),
  ],
});
```

W konfiguracji:

- `manifest` — z tych danych wtyczka utworzy plik `manifest.webmanifest` i doda odnośnik do niego w `index.html` (znaczenie pól jak w lab01),
- pliki samej aplikacji (HTML, JS, CSS, ikony) wtyczka zapisuje w cache automatycznie (**precache**),
- `runtimeCaching` — reguły dla zapytań wysyłanych w trakcie działania aplikacji:
  - **`NetworkFirst`** (najpierw sieć) dla API: service worker pyta serwer, a odpowiedź zapisuje w cache `weather-api`. Gdy sieci nie ma (albo serwer nie odpowie w ciągu 5 s — `networkTimeoutSeconds`), zwraca ostatnią zapisaną odpowiedź dla **tego samego adresu** (dlatego w `fetchWeather.js` zamieniamy nazwę miasta na małe litery). `expiration` ogranicza cache do 20 odpowiedzi, najwyżej sprzed doby,
  - **`CacheFirst`** (najpierw cache) dla ikon pogody — ikona zapisana raz jest potem brana z cache.

Service worker działa tylko w **wersji produkcyjnej** aplikacji — w trybie `npm run dev` wtyczka go nie uruchamia. Budujemy więc aplikację i uruchamiamy podgląd wersji produkcyjnej:

```
npm run build
npm run preview
```

Polecenie `build` tworzy folder `dist` z gotową aplikacją (to jego zawartość publikuje się na serwerze), a `preview` udostępnia go pod adresem `http://localhost:4173/`. Po każdej zmianie kodu te dwa polecenia trzeba powtórzyć (serwer `preview` zatrzymujemy **Ctrl+C**).

Test pracy offline:

1. Otwieramy `http://localhost:4173/`, a w DevTools → **Application** → **Service workers** sprawdzamy, czy service worker (`sw.js`) ma status **activated and is running**.
2. Wyszukujemy kilka miast (np. `Siedlce`, `Warszawa`). W **Application** → **Cache storage** pojawi się cache `weather-api` z zapisanymi odpowiedziami.
3. W zakładce **Network** zmieniamy **No throttling** na **Offline** i odświeżamy stronę (**F5**) — aplikacja nadal się wczytuje (pliki z precache).
4. Wyszukujemy `Siedlce` — widzimy kartę miasta z ostatnio pobranymi danymi (w **Network** przy zapytaniu do `api.openweathermap.org` w kolumnie **Size** widać `(ServiceWorker)` — odpowiedź

przyszła z cache). Wyszukujemy miasto, którego wcześniej nie szukaliśmy — pojawia się komunikat **Brak połączenia z internetem.**
5. Przywracamy **No throttling**.

Jeśli po zmianie kodu i ponownym `build` widzimy starą wersję, odświeżamy stronę jeszcze raz (nowy service worker instaluje się w tle) albo w **Application** → **Service workers** klikamy **Update**.

Przed zadaniami zatrzymujemy serwer `preview` (**Ctrl+C**) i wracamy do trybu deweloperskiego: `npm run dev`, adres `http://localhost:5173/`. Wersja `preview` pokazuje zbudowaną aplikację z folderu `dist` — zmian w kodzie w niej nie zobaczymy, dopóki nie zrobimy ponownie `npm run build`.

Zadania

Zadania wykonujemy w trybie `npm run dev`. Jeśli po wyszukaniu miasta strona zrobi się **pusta (biała)**, to w kodzie JSX jest błąd (np. literówka albo odwołanie do pola, którego nie ma) — jego opis znajdziemy w konsoli (**F12** → **Console**).

1. **Temperatura.** W karcie miasta, **pod nagłówkiem** `</h2>` (wewnątrz `<div className="city">`), wyświetl temperaturę (`weather.main.temp`, zaokrągloną do liczby całkowitej — `Math.round()`) w elemencie z klasą `city-temp`. Znak stopni wstawiamy jako `°C` albo `°C`, np. w elemencie `<sup>`.

2. **Pogoda.** Pod temperaturą dodaj element `<div className="info">`, a w nim:

   - ikonę pogody — element `<img>` z klasą `city-icon`, opisem w `alt` i adresem:

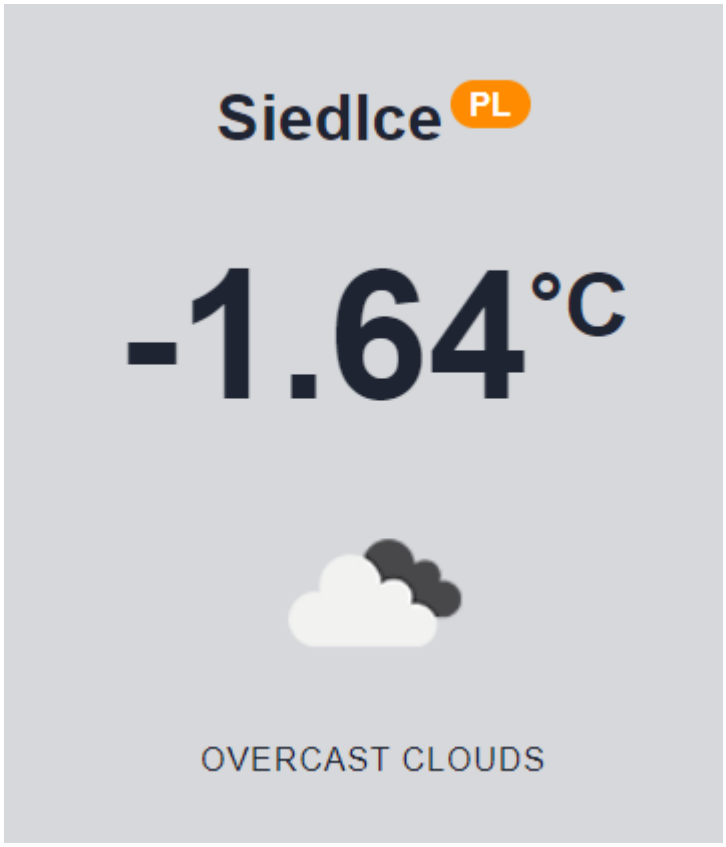
     ```
     src={`https://openweathermap.org/img/wn/${weather.weather[0].icon}@2x.png`}
     ```

   - opis pogody w elemencie `<p>`:

     ```
     <p>{weather.weather[0].description}</p>
     ```

   Pole `weather.weather` jest **tablicą** (zwykle z jednym elementem), dlatego odwołujemy się do pierwszego elementu: `[0]`.

   Przykładowy wynik (zadania 1 i 2):



Dodatkowo wyświetl:

- ciśnienie atmosferyczne (hPa),
- wilgotność (%),
- prędkość wiatru (m/s) oraz porywy wiatru — pole `wind.gust` **nie zawsze** jest w odpowiedzi; wyświetl je tylko wtedy, gdy istnieje,
- kierunek wiatru w stopniach (`wind.deg`; dla chętnych — jako strona świata, np. `NW`).

3. **Aktualność danych.** Wyświetl godzinę pomiaru pogody — pole `weather.dt` to liczba **sekund** od 1970 r., a `new Date()` oczekuje milisekund:

```
new Date(weather.dt * 1000).toLocaleTimeString('pl-PL', { hour: '2-digit', minute: '2-digit' })
```

Dodaj też informację o braku połączenia: stan `online` z wartością początkową `navigator.onLine` i nasłuchiwanie zdarzeń `online` / `offline` obiektu `window` (np. w `useEffect`). Gdy aplikacja jest offline, nad kartą wyświetl komunikat, np. **Brak połączenia — pokazuję ostatnio pobrane dane.** Sprawdź działanie w trybie offline (wersja `build` + `preview`).

4. **Instalacja.** Zbuduj aplikację i uruchom podgląd (`npm run build`, `npm run preview`). W DevTools → **Application** → **Manifest** sprawdź sekcję **Errors and warnings** — ostrzeżenia „Richer PWA Install UI won't be available…" (brak zrzutów ekranu `screenshots`) i notkę o braku pola `id` możemy zignorować, nie blokują instalacji. Zainstaluj aplikację (ikona instalacji w pasku adresu Chrome) i sprawdź, czy zainstalowana aplikacja działa bez połączenia (w jej oknie DevTools otwieramy **Ctrl+Shift+I**). Dla chętnych: usuń ostrzeżenia, dodając w `manifest` w `vite.config.js` pole `id: '/'` oraz listę `screenshots` (zrzut szeroki z `form_factor: 'wide'` i wąski bez `form_factor`).

Oddanie

Kod umieszczamy w repozytorium GitHub w folderze **`lab_03/`** — cały projekt poza folderami `node_modules` i `dist` (Vite dopisał je już do pliku `.gitignore`). W pliku `lab_03/README.md` opisujemy, jak uruchomić aplikację (`npm install`, `npm run build`, `npm run preview`), i dołączamy zrzuty ekranu: pogoda dla wybranego miasta oraz działanie w trybie offline.

Klucz API zapisany w kodzie będzie widoczny w publicznym repozytorium. W aplikacji działającej w przeglądarce i tak nie da się go ukryć (każdy może go podejrzeć w zakładce **Network**), a darmowy klucz OpenWeatherMap ma niski limit zapytań — dlatego nie jest to groźne. Klucze do płatnych usług trzyma się na serwerze, nigdy w kodzie aplikacji webowej.

Źródła:

- https://vite.dev/guide/
- https://react.dev/learn
- https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- https://openweathermap.org/current
- https://vite-pwa-org.netlify.app/guide/
- https://developer.chrome.com/docs/workbox/caching-strategies-overview